Formula Uno

Complete Architecture & Model Internals

The exhaustive reference — every module, every feature, every decision

This document is the definitive technical account of the system. It walks through the architecture layer by layer, the feature pipeline transformation by transformation, the mathematics of the learning algorithm, the simulation engine line by line, and the full history of everything that was built, modified, considered, and deliberately rejected.

Ten parts and three appendices. Every code reference reflects the current state of the source on disk; every quoted result comes from a verified run on real Bahrain data.

Contents

Part 0	Orientation — how to read this document
Part 1	System Architecture — the layered design
Part 2	The Data Layer — FastF1, schemas, caching, the era boundary
Part 3	Feature Engineering — every transformation and every feature
Part 4	Model Internals — gradient boosting from first principles
Part 5	The Race Engine — the simulation, line by line
Part 6	Analytics & Validation — how accuracy is measured
Part 7	The Orchestration Pipeline
Part 8	Complete Change Log — modified & considered
Part 9	Results — verified numbers and error decomposition
Part 10	Glossary
Appendix A	Feature reference card
Appendix B	Hyperparameter reference card
Appendix C	File-by-file responsibility matrix

Part 0 — Orientation

Formula Uno is a machine-learning system that reconstructs and predicts Formula 1 race results by learning how fast each car laps under given conditions, then replaying a Grand Prix one lap at a time from those learned lap times. This document explains that sentence in exhaustive detail.

0.1 The single-paragraph summary

> **NOTE — The whole system in one paragraph**
>
> Two **XGBoost gradient-boosted-tree regressors** — one trained on race laps, one on qualifying laps — each read ten engineered features (driver, team, circuit, year-in-era, lap number, tyre compound, tyre age, tyre age squared, fuel load, grid position) and output a single predicted lap time in seconds. A deterministic **race engine** then simulates a Grand Prix lap by lap: it asks the race model for each driver's lap time, accumulates those times, splices in real historical pit-stop laps, and after every lap re-sorts the field by total elapsed time so finishing order emerges rather than being predicted directly. Models train on 2022–2024 and are tested out-of-sample on an unseen 2025.

0.2 How to read this document

- **Parts 1–2** give the big picture and the data foundation — read these first.
- **Parts 3–5** are the technical core: features, the model mathematics, and the simulator. This is where the depth lives.
- **Parts 6–7** cover validation and orchestration.
- **Part 8** is the complete history of what was changed and what was considered — the "why it looks like this" chapter.
- **Parts 9–10 and the appendices** are reference material: results, definitions, and quick-lookup cards.

Part 1 — System Architecture

1.1 The layered design philosophy

The system is built as a one-directional pipeline of independent layers. Each layer has a single responsibility and communicates with the next only through well-defined data structures (pandas DataFrames and plain dictionaries). This separation is deliberate: it means each layer can be understood, tested and validated on its own, and a change in one rarely ripples into another.

1.2 The module map

The repository is organised as a small Python package under **src/**, with thin runner scripts at the root that wire it together for specific circuits.

```
Formula Uno/
|- run_bahrain.py          entry point: Bahrain, 2022-24 train -&gt; 2025 test
|- run_spa.py              entry point: Spa
|- run_multi_circuit.py    entry point: combined multi-circuit model
|- (run_features / run_model / run_engine / run_all .py  -- legacy step runners)
|- data/                   FastF1 cache + data/models/*.pkl saved models
|- src/
   |- ingestion/load_session.py    load race/qualifying sessions from FastF1
   |- features/build_features.py   raw laps -&gt; clean numeric feature table
   |- model/train.py               XGBoost training, CV, save/load, importance
   |- engine/race_engine.py        lap-by-lap race simulation
   |- analytics/compare.py         simulated vs actual, error metrics
   |- analytics/qualifying.py      qualifying extraction, prediction, report
   |- pipeline.py                  orchestrates everything (run_circuit etc.)
```

1.3 The end-to-end data flow

A single circuit run flows through the system as follows. Every arrow is a data hand-off; the data structure passed is named in parentheses.

```
FastF1 API
   | (race + qualifying sessions, per year)
   v
load_session.py  -&gt;  {"laps": DataFrame, "results": DataFrame}  per session
   |
   v
build_encoders()  -&gt;  {Driver, Team, Track} LabelEncoders  (fit on ALL sessions)
   |
   v
build_feature_frame()          build_qualifying_feature_frame()
   | (race feature table)        | (qualifying feature table)
   v                             v
train() race model             train() qualifying model     -&gt; data/models/*.pkl
   |                             |
   v                             v
RaceEngine.simulate()          predict_qualifying()
   | (per-lap history DataFrame) | (per-driver prediction table)
   v                             v
compare.print_summary()        qualifying.print_qualifying_report()
   | position error + lap RMSE   | grid error + lap RMSE
```

1.4 The two-phase lifecycle

Conceptually every run has two phases with a hard boundary between them:

- **Learning phase** (probabilistic): the encoders are fit and the two XGBoost models are trained. This is the only part of the system that “learns” anything from data.
- **Simulation phase** (deterministic): the trained models are treated as fixed oracles. The race engine and the analytics are ordinary procedural code — given the same models and inputs they produce identical output every time.

> **DESIGN POINT — Why the learned/deterministic boundary matters**
>
> Keeping the learned component small and well-contained (just two models, each predicting one lap time) means the system’s behaviour is almost entirely inspectable. Any surprising race outcome can be traced either to a lap-time prediction (the model) or to the simulation logic (plain code) — never to some entangled mixture. This is a core design principle, not an accident.

1.5 Design principles enforced throughout

Principle	What it means in practice
Separation of learning and simulation	The model predicts pace; the engine turns pace into results. Neither knows the other’s internals.
Training-serving symmetry	The exact same encoders and feature formulas are used at training time and at prediction time.
Falsifiability	The system only makes claims reality can check — reconstructing real races, never untestable hypotheticals.
Out-of-sample honesty	The test year is never seen during training, in any form, by any component.
Physical interpretability	Every feature corresponds to a real, nameable physical quantity.

Part 2 — The Data Layer

2.1 FastF1

All data originates from **FastF1**, an open-source library that exposes Formula 1's official timing feed. It provides, per session, a table of every lap every driver completed and a table of the classified results. The project consumes only lap-level and result-level data; telemetry, weather and race-control message streams are switched off at load time to keep each session light and the cache small.

2.2 load_session.py — line walkthrough

```
fastf1.Cache.enable_cache("data")           # module import: turn on disk cache

def _load(year, event, session_type):
    session = fastf1.get_session(year, event, session_type)   # e.g. 2024,"Bahrain Grand Prix","R"
    session.load(telemetry=False, weather=False, messages=False)
    return {"session": session,
            "laps":    session.laps.copy(),
            "results": session.results.copy()}

def load_race(year, event):        return _load(year, event, "R")
def load_qualifying(year, event):  return _load(year, event, "Q")
```

The key subtlety is the **event** argument: FastF1 accepts either an integer round number or a string event name. This flexibility is convenient but was the source of a serious bug (see Part 8) — round numbers are not stable across seasons.

2.3 The laps DataFrame — the columns that matter

FastF1 returns many columns per lap. These are the ones the system reads or filters on:

Column	Type	How the system uses it
Driver	str (3-letter)	Encoded to an integer feature; also the grouping key
Team	str	Encoded to an integer feature
LapNumber	int	Feature; drives the fuel formula and the simulation loop index
LapTime	timedelta	Converted to seconds — this is the prediction TARGET
Compound	str	Ordinal-encoded feature (SOFT..WET)
TyreLife	float	Feature; also squared into TyreLifeSquared
Position	float	Filter only (a lap must have a running position to be valid)
PitInTime	timedelta/NaT	Presence marks a pit in-lap; builds the pit schedule
IsAccurate	bool	Data-quality filter — only accurate laps train the model
Deleted	bool/None	Filter — deleted (e.g. track-limits) laps are dropped
TrackStatus	str codes	Filter — Safety Car (4) / VSC (6) laps are dropped

2.4 The results DataFrame

The results table supplies two things the laps table does not carry cleanly: each driver's **Abbreviation** (their 3-letter code, joined to laps) and **GridPosition** (their starting slot, used both as a model feature and to initialise the simulation). Position in the results table gives the official classified finishing order used by the analytics for comparison.

2.5 The caching mechanism

On first request FastF1 downloads a session and writes it to **data/**. Every later request for the same session reads from disk in milliseconds. This is why run logs show “Using cached data” for everything except genuinely new sessions. Practically, it means the expensive network step happens once per session ever, and all subsequent iteration on features or the model is fast.

2.6 The int-vs-string event trap

CAVEAT — Round numbers are not stable across seasons

Addressing a race by round number (event=1) returns whatever race was first on that season’s calendar. Round 1 was Bahrain in 2022–2024 but **Australia in 2025**. Using the round number therefore silently mixed circuits. The fix — and the standing rule — is to address races by name (event=“Bahrain Grand Prix”), which FastF1 resolves correctly in every season. Full story in Part 8.

2.7 The regulation-era boundary (2022)

The training window begins in 2022 because that season introduced a complete aerodynamic rules reset — the ground-effect era. The 2022-onward cars generate downforce through shaped underfloor tunnels rather than primarily over-body surfaces, so their lap-time behaviour as a function of speed, fuel and tyre state is governed by different physics than the 2017–2021 generation.

NOTE — Consequence for training data

A pre-2022 lap is a sample from a different physical regime. Including such laps would teach the model a car behaviour that no longer exists, biasing every prediction. 2022 is therefore treated as **Year 0** of the era, encoded directly as the YearEncoded feature so the model can still learn how performance develops *within* the era without ever mixing in obsolete physics.

Part 3 — Feature Engineering

This is the layer that converts messy, human-readable lap records into a clean numeric matrix an algorithm can learn from. It lives in **build_features.py**. Every step below is a deliberate transformation; nothing is incidental.

3.1 The constants

```
COMPOUND_ORDER = {"SOFT":0,"MEDIUM":1,"HARD":2,"INTERMEDIATE":3,"WET":4}
REG_ERA_START  = 2022     # ground-effect era = Year 0
START_FUEL_KG  = 105.0    # typical race start fuel load
QUAL_FUEL_KG   = 5.0      # qualifying runs light

FEATURE_COLS = ["DriverEncoded","TeamEncoded","TrackEncoded","YearEncoded",
                "LapNumber","CompoundEncoded","TyreLife","TyreLifeSquared",
                "FuelLoad","GridPosition"]
TARGET_COL   = "LapTimeSeconds"
```

3.2 The encoder construction — build_encoders()

Before any feature frame is built, the categorical encoders are fitted **once**, on the union of every session that will ever be used (all training years plus the test year, race and qualifying). Each unique driver, team and track name is assigned a stable integer.

```
def build_encoders(sessions):
    all_drivers = pd.concat([s["laps"]["Driver"].dropna() for s in sessions]).unique()
    all_teams   = pd.concat([s["laps"]["Team"].dropna()   for s in sessions]).unique()
    all_tracks  = [s["track_name"] for s in sessions]
    return {"Driver": LabelEncoder().fit(sorted(all_drivers)),
            "Team":   LabelEncoder().fit(sorted(all_teams)),
            "Track":  LabelEncoder().fit(sorted(set(all_tracks)))}
```

DESIGN POINT — The unseen-label problem this solves

A LabelEncoder can only transform values it was fitted on. If it were fitted only on training years, a driver who debuts in the test year (a 2025 rookie) or a renamed team (Alfa Romeo → Kick Sauber) would be an unknown value and raise a ValueError at prediction time. Fitting on the union guarantees every code that can ever appear already exists. The **same** fitted encoder objects are then reused everywhere — this is training-serving symmetry in action.

3.3 The race feature pipeline — build_feature_frame()

The race pipeline applies its transformations in a fixed order. Note especially that the true race length is captured *before* any laps are filtered out, so the fuel formula uses the real distance:

```
laps = _lap_time_to_seconds(laps)
total_laps = int(laps["LapNumber"].max())   # BEFORE filtering: true race distance
laps = _drop_bad_laps(laps)
laps = _encode_compound(laps)
laps = _encode_categoricals(laps, encoders)
laps = _merge_grid_position(laps, results)
laps = _encode_track(laps, track_name, encoders["Track"])
laps["YearEncoded"]     = float(year - REG_ERA_START)
laps["FuelLoad"]        = START_FUEL_KG - (laps["LapNumber"] - 1) * (START_FUEL_KG / total_laps)
laps["TyreLifeSquared"] = laps["TyreLife"] ** 2
```

Step: _lap_time_to_seconds

Converts the timedelta LapTime column into a floating-point number of seconds (LapTimeSeconds), which is the regression target.

Step: _drop_bad_laps — every predicate

Keeps only laps that are trustworthy pace samples. A lap survives only if **all** of these hold:

- LapTimeSeconds is present (not null).
- IsAccurate is true — FastF1's own data-quality flag.
- Deleted is false — computed as **~laps["Deleted"].fillna(False).astype(bool)** so that the None values FastF1 stores for non-deleted laps become False before inversion.
- LapNumber, TyreLife, Compound and Position are all present.
- Compound is not the string "UNKNOWN".
- TrackStatus does not contain code 4 (Safety Car) or 6 (Virtual Safety Car) — those laps are artificially slow and would corrupt the model's notion of green-track pace.

The pit in-lap and out-lap are naturally excluded too, because their times are dominated by pit-lane transit rather than pace; the engine handles them separately with real historical times.

Step: _encode_compound

Maps the compound string to an ordinal integer via COMPOUND_ORDER. The ordering is meaningful here (softer compounds are grippier but wear faster), so an ordinal code is appropriate — unlike driver or team, where the integers are arbitrary labels.

Step: _encode_categoricals

Applies the pre-fitted Driver and Team encoders. It never refits — refitting here would break training-serving symmetry and could assign a driver a different integer at prediction time than at training time, silently corrupting results.

Steps: _encode_track and _merge_grid_position

_encode_track stamps the circuit's integer code onto every lap. _merge_grid_position joins each driver's starting position from the results table onto their laps, coercing it to a number.

Derived columns: YearEncoded, FuelLoad, TyreLifeSquared

YearEncoded is *year – 2022*. FuelLoad is the linear fuel-burn estimate in kilograms. TyreLifeSquared is simply TyreLife squared. The frame returned contains the ten feature columns plus the target and a few display columns (Driver, Team, Compound, Position) for reporting.

3.4 Every feature explained in depth

Feature	Encoding	What it represents / why it helps
DriverEncoded	Label (arbitrary int)	Per-driver pace signature learned from history
TeamEncoded	Label (arbitrary int)	Per-constructor car-performance signature
TrackEncoded	Label (arbitrary int)	Circuit identity — anchors absolute lap-time scale
YearEncoded	year – 2022	Position in the era; lets the model learn development trends
LapNumber	raw int	Track evolution (rubbering-in); simulation loop index
CompoundEncoded	Ordinal 0–4	Tyre type; softer = grippier but shorter-lived
TyreLife	raw int	Laps on the current set — the primary degradation signal
TyreLifeSquared	TyreLife²	Curvature — models accelerating wear toward the cliff
FuelLoad	105 – burn	Fuel mass in kg; heavier = slower (~0.03 s/kg, learned)

Feature	Encoding	What it represents / why it helps
GridPosition	raw int	Proxy for weekend car/driver quality

3.5 The qualifying feature pipeline — build_qualifying_feature_frame()

The qualifying pipeline mirrors the race one with three deliberate differences, because a qualifying lap is a physically different event:

- **FuelLoad is fixed at ~5 kg** (QUAL_FUEL_KG) — qualifying cars run minimal fuel for a single flying lap, so there is no burn-down curve.
- **The Position filter is dropped** — a qualifying lap has no running race position, so _drop_bad_qual_laps omits that predicate while keeping all the others (including the SC/VSC filter).
- **GridPosition comes from the qualifying result's own position** rather than the race grid, and drivers or teams outside the encoder universe are filtered out defensively before encoding.

3.6 The FuelLoad collinearity analysis

A precise technical point worth stating carefully. Within a single race, FuelLoad is an exact affine function of LapNumber:

$$\text{FuelLoad} = 105 - (\text{LapNumber} - 1) \times (105 / \text{total_laps})$$
$$= a - b \cdot \text{LapNumber} \quad (a, b \text{ constant for a fixed race length})$$

So for a single circuit the two features are perfectly collinear — a decision tree can split on either equivalently, and FuelLoad adds little new information. Its genuine value is **cross-circuit**: because the slope b depends on race length, the same lap number maps to different fuel masses at different tracks (70.0 kg on lap 20 of 57 at Bahrain versus 59.7 kg on lap 20 of 44 at Spa). It thus re-expresses race progress on a common physical scale, which is what lets a combined multi-circuit model transfer fuel and tyre knowledge between circuits of different lengths.

3.7 What was deliberately NOT included as a feature

Several plausible inputs were left out on purpose, each for a concrete reason:

Excluded input	Why it was left out
Weather / track temperature	Loaded data has weather disabled for speed; adding it is a clear future extension but increases variance and 2025 conditions differ per session.
Telemetry (speed traces, throttle)	Far heavier data; the project targets lap-level modelling, not corner-level.
DRS / slipstream / traffic	Not cleanly available per lap; would require modelling car-to-car interaction the simulator does not attempt.
Driver ratings	Rejected on principle — encodes author bias and makes results circular (Part 8).
Pit-lane / in-lap times as features	Handled as real historical values in the engine, not learned, to keep strategy and pace validation separate.

Part 4 — Model Internals

Both models are **XGBoost regressors** — there are no neural networks anywhere in the system. This part explains what that means, from the ground up and then into the mathematics.

4.1 The ladder of ideas: tree → boosting → XGBoost

- **Decision tree.** A hierarchy of yes/no tests on features (“TyreLife > 15?”, “compound = HARD?”) ending in leaves that each output a number. Splits are chosen to reduce prediction error. One tree alone is weak — it either oversimplifies or memorises noise.
- **Ensemble.** Combine many trees so their individual errors cancel.
- **Bagging vs boosting.** A Random Forest grows many independent trees in parallel and averages them (bagging). Boosting grows trees in sequence, each correcting the residual error left by the ones before it.
- **Gradient boosting.** Each new tree is fitted to the negative gradient of the loss — for squared error, that is exactly the residual (actual minus current prediction). The ensemble is the sum of all trees, each scaled by a small learning rate.
- **XGBoost.** A fast, regularised implementation of gradient boosting with second-order optimisation and built-in overfitting controls. It is the standard tool for tabular problems like this.

4.2 The mathematics of gradient boosting

The prediction is an additive ensemble of M regression trees, each scaled by the learning rate η:

$$\hat{y}_i = F_M(x_i) = \sum_{m=1..M} \eta \cdot f_m(x_i)$$

Training minimises a regularised objective: a loss term measuring fit to the data, plus a complexity penalty on each tree that discourages overfitting:

$$L = \sum_i l(y_i, \hat{y}_i) + \sum_m \Omega(f_m)$$
$$\Omega(f) = \gamma T + \tfrac{1}{2} \lambda \sum_j w_j^2$$

Here T is the number of leaves in a tree and w_j the value at leaf j; γ penalises adding leaves and λ penalises large leaf values. XGBoost adds each new tree by taking a second-order Taylor expansion of the loss around the current prediction, using the gradient g_i and Hessian h_i of the loss at each data point:

$$g_i = \partial l / \partial \hat{y}_i \qquad h_i = \partial^2 l / \partial \hat{y}_i^2$$

For the squared-error loss used here, $l = (y_i - \hat{y}_i)^2$, so $g_i = 2(\hat{y}_i - y_i)$ — proportional to the residual — and $h_i = 2$. Given a fixed tree structure, the optimal value for leaf j and the quality (gain) of a candidate split follow in closed form:

$$w_j^* = -\left(\sum_{i \in j} g_i\right) / \left(\sum_{i \in j} h_i + \lambda\right)$$

The tree-building algorithm greedily chooses the split that most increases this gain, stopping when no split improves the penalised objective. In plain terms: **each tree asks where the current ensemble is still wrong and by how much, and adds a small, complexity-controlled correction.**

4.3 Every hyperparameter, in depth

Hyperparameter	Value	Meaning, and the trade-off it controls
n_estimators	2000	Number of boosting rounds (trees). More trees fit finer detail but cost time and, past a point, overfit. Paired with a small learning rate.

Hyperparameter	Value	Meaning, and the trade-off it controls
learning_rate	0.02	Shrinkage η applied to each tree. Small = careful steps = better optimum, but needs many trees to converge.
max_depth	6	Maximum depth of each tree. Deeper = captures higher-order feature interactions but risks overfitting. Six is a balanced mid value.
min_child_weight	5	Minimum summed Hessian (≈ number of laps) required behind a leaf. Higher = leaves must be backed by real evidence = less overfitting to noise.
subsample	0.8	Fraction of rows each tree sees. Row-level randomness (stochastic gradient boosting) improves robustness and generalisation.
colsample_bytree	0.8	Fraction of features each tree sees. De-correlates the trees, reducing variance.
random_state	42	Seeds all randomness so a run is exactly reproducible.
n_jobs	-1	Use all CPU cores for training.

4.4 The two models side by side

	Race-pace model	Qualifying model
Algorithm	XGBoost (identical config)	XGBoost (identical config)
Feature set	The same 10 features	The same 10 features
Training rows	~2,831 race laps (2022–24)	~253 qualifying laps (2022–24)
Fuel feature	105 kg burning down	fixed ~5 kg
CV RMSE (Bahrain)	~0.79 s	~4.4 s (± ~2.2 — small-data variance)
Purpose	Reconstruct the full race	Predict the grid order

> **CAVEAT — The qualifying model's honest limitation**
>
> With only a handful of clean flying laps per driver per weekend, the qualifying training set is small (~253 laps) and its cross-validation error is correspondingly high and variable. It nonetheless predicted the 2025 Bahrain pole time to within ~0.11 s, because single-lap pace at a fixed circuit is quite stable year to year — but the small-data caveat is real and should be stated whenever the qualifying numbers are quoted.

4.5 Cross-validation and the overfitting check

Before fitting the final model, train() runs 5-fold cross-validation: the laps are split into five parts; the model trains on four and is scored on the held-out fifth; this rotates through all five and the errors are averaged. The resulting **CV RMSE** estimates performance on unseen data honestly, before any race is simulated. The final model is then fitted on all rows, and its **training** MAE and RMSE are reported. A training error far below the CV error would signal overfitting; here the gap is modest and expected.

4.6 Serialization and inspection

Trained models are pickled to **data/models/{name}.pkl** and can be reloaded without retraining. feature_importance() prints each feature's gain-based importance, ranking which inputs the model actually relies on — a transparency a neural network would not offer.

4.7 Why XGBoost and not a neural network

- **Small tabular data.** ~2,831 (and ~253) rows; neural nets need far more and overfit here. Trees excel on small/medium tabular sets.
- **Mixed feature types, no spatial/sequential structure** for a CNN or RNN to exploit.

- **Automatic feature interactions** via split chains — no manual crossing needed.
- **Interpretability** through gain scores — essential for a project one must explain.
- **Practicality** — no GPU, seconds to train, insensitive to feature scaling.
- **Empirical evidence** — a published six-model bake-off on this exact problem found the neural network worst and tree ensembles best.

Part 5 — The Race Engine

The engine (race_engine.py) turns the race-pace model into a full Grand Prix. It is the most intricate module and is entirely deterministic — given the same model it produces the same race every time.

5.1 DriverState — the per-car state object

Each car is a mutable dataclass that evolves lap by lap:

```
@dataclass
class DriverState:
    name; team; driver_encoded; team_encoded   # identity (fixed)
    grid_position                               # starting slot (fixed)
    compound; tyre_age                          # tyre state (mutates on pit stops)
    cumulative_time = 0.0                       # running total elapsed time
    position = 0                                # current running position
    lap_times = []; positions = []              # per-lap history
    retired = False                             # ghost-car flag (see 5.7)
```

5.2 Initialisation — _build_driver_states()

For each driver in the real race, the engine reads their actual starting compound, starting tyre age and grid position, and builds a DriverState. Notable details:

- The **starting compound** is the first non-null compound the driver ran; if somehow absent it defaults to MEDIUM.
- The **starting tyre age** is FastF1's lap-1 TyreLife, which already accounts for warm-up and installation laps run before the start.
- If a driver or team is not in the encoder universe, the driver is **skipped** (a defensive guard; in normal runs the union-fitted encoders cover everyone).
- Grid position defaults to 20 if missing, so a pit-lane start does not crash initialisation.

5.3 The pit schedule — _build_pit_schedule()

Before racing, the engine scans the real laps for pit stops (rows with a PitInTime) and records, per driver and per in-lap: the new compound fitted, the real in-lap time, the out-lap number, and the real out-lap time.

> **NOTE — Why pit laps use real times rather than the model**
>
> The model was never trained on in-laps and out-laps — they were filtered out as unrepresentative of pace. Predicting them would be asking the model about something it never learned. Using the real historical times instead cleanly separates *strategy* accuracy from *speed-model* accuracy, so each can be judged independently.

5.4 Single-lap inference — _predict_lap()

For a normal racing lap the engine assembles the exact same ten features used in training, in the same order, recomputing fuel load and squared tyre life on the fly so training and simulation stay identical:

```
fuel_load    = 105.0 - (lap_number - 1) * (105.0 / total_laps)
tyre_life_sq = state.tyre_age ** 2
features = {DriverEncoded, TeamEncoded, TrackEncoded, YearEncoded, LapNumber,
            CompoundEncoded, TyreLife, TyreLifeSquared, FuelLoad, GridPosition}
return float(model.predict(DataFrame([features])[feature_cols])[0])
```

5.5 The simulation loop — simulate()

The heart of the engine. Race length is the maximum lap number in the real data (optionally capped by max_laps for a short validation run). Then, for every lap, for every non-retired driver, exactly one of three cases fires:

1. **Out-lap** — the driver is on the lap after a stop (tracked in out_lap_queue). Use the real historical out-lap time. If it is somehow missing, fall back to a predicted lap plus 15 s.
2. **In-lap** — the driver pits at the end of this lap. Use the real historical in-lap time (fallback: predicted plus 20 s). Then switch the car to its new compound, reset tyre age to zero, and queue the out-lap for next lap.
3. **Normal lap** — call _predict_lap for a model-predicted lap time.

After the lap time is determined, the driver's state updates: cumulative time increases, tyre age increments, and the lap time is recorded. Once every driver has completed the lap, the field is sorted:

```
active = [(d, s.cumulative_time) for d,s in states.items() if not s.retired]
active.sort(key=lambda x: x[1])          # lowest total time = leader
pos_map = {driver: rank+1 for rank,(driver,_) in enumerate(active)}
```

DESIGN POINT — Positions emerge; they are never set

This is the central modelling choice of the whole project. No code ever decides "driver X finishes P3." Positions are purely a by-product of sorting cumulative times after each lap. A car that laps two tenths quicker every lap climbs the order on its own, exactly as in a real race. The finishing order is simply the sort at the final lap.

Every tenth lap (and the final lap) the engine prints the leader and the gap to second place, giving a live sense of how the reconstructed race unfolds.

5.6 The ghost-car limitation

CAVEAT — Retired and disqualified cars keep circulating

The engine runs every car for the full distance; there is no retirement or mechanical-failure model. A driver who in reality retired or was disqualified still accumulates predicted lap times — a "ghost car" — and can therefore appear mid-order in the simulation where reality classified them last. This is a deliberate, understood simplification: the system reconstructs **pace**, and mechanical or administrative outcomes are out of scope. The clearest example is George Russell at Spa 2024, whom the model correctly rated at winning pace but who was disqualified post-race for an underweight car; the simulation shows his pace, not his classification.

5.7 The output

simulate() returns a tidy DataFrame with one row per (driver, lap): the lap time, cumulative time, compound, tyre age, grid position and running position. Everything the analytics layer needs to score the reconstruction is contained in this single table.

Part 6 — Analytics & Validation

The analytics layer (compare.py and qualifying.py) scores a reconstruction against reality using two independent metrics, because a single number would hide as much as it reveals.

6.1 Position accuracy — compare_positions()

final_classification() takes the simulation's last lap and reads off the finishing order. compare_positions() joins that against the official classified result per driver and computes the absolute position error. print_summary() then reports three headline numbers: exact positions correct, positions within two, and mean absolute position error.

6.2 Lap-time accuracy — lap_time_rmse()

Independently, each simulated lap is matched to the corresponding real lap (on driver and lap number), restricted to clean (IsAccurate) laps so the comparison uses the same population the model trained on. The per-driver RMSE and the aggregate **Fleet RMSE** measure raw pace accuracy in seconds, decoupled from finishing order.

> **NOTE — Why report two numbers, not one**
>
> Lap-time accuracy and position accuracy can diverge. Midfield cars are separated by hundredths of a second, so a model can predict every lap time almost perfectly yet still shuffle a few positions; conversely it can land positions while individual laps drift. Reporting both — pace error *and* order error — tells a far more honest story than either alone.

6.3 gap_evolution()

A diagnostic that returns the time gap between any two drivers across every lap, useful for inspecting how a battle or a pit-stop undercut plays out in the reconstruction.

6.4 Qualifying validation

extract_best_qualifying_laps() finds each driver's single fastest clean qualifying lap — the lap that set their real grid slot. predict_qualifying() runs a model over those laps' features (with fuel fixed at 5 kg) and ranks drivers by predicted time to produce a predicted grid, compared against the real grid. The function is model-agnostic; the pipeline passes it the dedicated qualifying model. print_qualifying_report() summarises grid accuracy and lap-time RMSE.

6.5 In-sample versus out-of-sample

Every training year is simulated back as an **in-sample** sanity check — the model should be able to reproduce races it learned from, and improving in-sample accuracy year over year confirms it is genuinely learning the era. The test year is simulated as the **out-of-sample** test — the real measure of whether learned pace generalises to a season never seen.

Part 7 — The Orchestration Pipeline

pipeline.py wires every layer together. It exposes two entry points.

7.1 run_circuit() — one circuit

The full sequence for a single track:

1. Load every training-year race and qualifying session, plus the test-year race and qualifying.
2. Build shared encoders across all of those sessions (race + qualifying, every year).
3. Build the race feature table (concatenated across training years) and train the race model; save it.
4. Build the qualifying feature table and train the dedicated qualifying model; save it.
5. Simulate every training year in-sample and print a summary for each.
6. Simulate the test year out-of-sample and print its summary.
7. Extract the test-year best qualifying laps, predict them with the qualifying model, and print the qualifying report.

7.2 run_multi_circuit() — combined model

Loads several circuits at once, builds one shared encoder set spanning all of them, concatenates every circuit's laps into a single training set, and trains **one** race model and **one** qualifying model across all circuits. It then reports per circuit. This is where the FuelLoad feature earns its keep, because the combined model must reason across circuits of different lengths.

7.3 The runner scripts

run_bahrain.py, run_spa.py and run_multi_circuit.py are three-line entry points that call the pipeline with the right event names and years. They address races **by name**, not round number. (A set of older run_*.py scripts from earlier development still exist as per-layer step runners.)

Part 8 — Complete Change Log

Everything that was built, modified, or considered — including the ideas that were deliberately rejected. This is the “why it looks like this” chapter.

8.1 Chronological build history

Phase	What was built
1 — Foundations	FastF1 ingestion; first 7-feature pipeline; lap-time target.
2 — Race engine	Lap-by-lap simulator; positions from cumulative time; real pit times.
3 — Cross-year model	Trained across 2022–24 with YearEncoded; union-fitted encoders; test on 2025.
4 — Domain corrections	Reconciled model pace vs administrative results (e.g. Russell DSQ).
5 — Accuracy overhaul	Fuel load, TyreLife², SC/VSC filter, tuned hyperparameters, dedicated qualifying model; fixed the circuit-name bug.

8.2 Every bug and its fix

Issue	Root cause	Fix
TypeError: unary ~ on None	FastF1 stores None, not False, for non-deleted laps	~laps[Deleted].fillna(False).astype(bool)
Feature-count mismatch (8 vs 7)	LapNumber listed in both feature and display columns → duplicated	Removed from display list; deleted stale model; retrained
UnicodeEncodeError on arrow	Windows console could not encode the → glyph	Replaced → with -> in all prints
ValueError: unseen label	Encoder fitted only on training years; new drivers/teams in test year	Fit encoders on the union of all sessions
KeyError: YearEncoded	Qualifying path built features without YearEncoded	Added year params + YearEncoded; threaded test_year through pipeline calls
Wrong circuit entirely	event=1 = Bahrain in 22–24 but Australia in 25	Address races by name; delete stale models; retrain

8.3 Design decisions considered — and the verdict

Considered	Verdict	Reasoning
Driver/team rating system	Rejected	Encodes author bias; makes output circular and unfalsifiable.
Hypothetical scenario simulator	Rejected	No ground truth; predictions cannot be checked against reality.
Single training year	Replaced	Cannot capture year-on-year car development; extended to 2022–24 + YearEncoded.
One model for race + qualifying	Replaced	Race fuel/pace regime is wrong for qualifying; split into two models.
Round-number event id	Replaced	Not stable across seasons; switched to event names.
Pre-2022 training data	Rejected	Different aero regulations — different physics would corrupt the model.
Weather / telemetry features	Deferred	Reasonable future extensions; excluded now for scope and variance.

8.4 The accuracy-improvement campaign

Improvement	Mechanism	Effect
Fuel load feature	Fuel mass in kg, learned s/kg	Cleaner pace signal; biggest gains in multi-circuit model
TyreLife squared	Curvature term for degradation	Fits the accelerating wear curve with less data
SC/VSC lap filter	Drop TrackStatus 4 and 6	Removes artificially slow outliers from training
Hyperparameter tuning	2000 trees @ 0.02, min_child_weight=5	Better optimum, less overfitting to noise
Dedicated qualifying model	Train on qualifying laps, light fuel	Predicted times on correct absolute scale (~1 s RMSE)
Circuit-name fix	event by name not round number	Corrected 2025 test from wrong circuit; RMSE 5.9 s → 1.5 s

Part 9 — Results (Verified)

All figures below come from a run on real Bahrain data (train 2022–24, test 2025) that was executed and reproduced before writing.

9.1 In-sample progression

Year (in-sample)	Mean position error	Fleet lap-time RMSE	Exact positions
2022	4.60 places	0.272 s	2 / 20
2023	2.80 places	0.247 s	2 / 20
2024	0.50 places	0.162 s	12 / 20

Accuracy sharpens year over year as more era data accumulates — evidence the model is genuinely learning the ground-effect-era pecking order rather than memorising one season.

9.2 Out-of-sample 2025 — the real test

Metric	Value
Mean position error	2.90 places
Within 2 positions	12 / 20
Fleet lap-time RMSE	1.492 s
Exact positions	3 / 20

9.3 Qualifying 2025

Metric	Value
Pole prediction (Piastri)	actual 89.841 s vs predicted 89.954 s (Δ +0.113 s)
Lap-time RMSE	1.009 s
Within 2 grid positions	15 / 20
Mean grid position error	1.90 places

9.4 Where the residual error comes from

The out-of-sample error is not random noise; it clusters on exactly the situations the model could not have learned:

- **Team changes** — a driver read partly through the pace of the car they used to drive (e.g. a Ferrari driver moving to Williams).
- **Rookies** with little or no prior race data in 2022–24, for whom the model has no pace history and must fall back on nearby signals such as grid position.

> **NOTE — The data-availability ceiling**
>
> No feature can conjure information that was never recorded. For a driver who never raced in the training window, the model has zero laps to learn from — the only real remedies are external data (pre-season testing, junior-formula results). This is a limit of the data, not a flaw in the method, and it accounts for most of the gap between the in-sample and out-of-sample numbers.

Part 10 — Glossary

XGBoost	Optimised, regularised gradient-boosted decision trees; the learning algorithm behind both models.
Gradient boosting	Building trees in sequence, each fitted to the residual error of the ensemble so far.
Decision tree	A flowchart of yes/no feature tests ending in numeric leaf predictions.
Bagging / Random Forest	Averaging many independent trees; contrasted with boosting's sequential correction.
Regularisation	Penalty terms (γ, λ) that discourage complex trees and large leaf values to curb overfitting.
Gradient / Hessian	First and second derivatives of the loss; XGBoost uses both to choose splits and leaf values.
Learning rate (η)	Shrinkage applied to each tree; small values need more trees but reach a better optimum.
RMSE	Root-mean-square error, in seconds; penalises large misses heavily. Always $\geq$ MAE.
Cross-validation	Rotating hold-out folds to estimate unseen-data performance before simulation.
In-sample / out-of-sample	Data the model did / did not train on; 2025 is the out-of-sample test.
LabelEncoder	Maps text categories (driver, team, track) to integers.
Ordinal encoding	Integer codes whose order carries meaning (tyre compounds), unlike arbitrary labels.
Training-serving symmetry	Using identical transforms at training and prediction time.
Feature / Target	A model input / the value it predicts (lap time in seconds).
Ground effect	The 2022 aero regulation defining the current era; the reason training starts in 2022.
YearEncoded	year – 2022; the lap's position within the regulation era.
Degradation / cliff	Gradual then accelerating loss of tyre grip; modelled via TyreLife and its square.
Fuel-corrected pace	Lap time adjusted for fuel mass; here fuel is a feature, its effect learned.
In-lap / out-lap	Laps entering / leaving the pits; handled with real times, not the model.
Cumulative time	Running total of a car's lap times; sorting it yields position.
Ghost car	A retired/disqualified car the simulation keeps running, since only pace is modelled.
Fleet RMSE	Lap-time RMSE aggregated across all drivers in a race.

Appendix A — Feature Reference Card

#	Feature	Encoding	Source
1	DriverEncoded	Label int	laps.Driver
2	TeamEncoded	Label int	laps.Team
3	TrackEncoded	Label int	track_label
4	YearEncoded	year – 2022	session year
5	LapNumber	raw int	laps.LapNumber
6	CompoundEncoded	Ordinal 0–4	laps.Compound
7	TyreLife	raw int	laps.TyreLife
8	TyreLifeSquared	TyreLife²	derived
9	FuelLoad	105 – burn (kg)	derived from LapNumber + race length
10	GridPosition	raw int	results.GridPosition (race) / qualifying position
T	LapTimeSeconds (target)	seconds	laps.LapTime

Appendix B — Hyperparameter Reference Card

Hyperparameter	Value	One-line role
n_estimators	2000	number of boosting trees
learning_rate	0.02	per-tree shrinkage η
max_depth	6	tree complexity
min_child_weight	5	minimum evidence per leaf
subsample	0.8	row fraction per tree
colsample_bytree	0.8	feature fraction per tree
random_state	42	reproducibility seed
n_jobs	-1	use all CPU cores

Appendix C — File Responsibility Matrix

File	Responsibility
ingestion/load_session.py	Load race/qualifying sessions from FastF1; enable cache
features/build_features.py	Encoders; race and qualifying feature pipelines; constants
model/train.py	XGBoost training, 5-fold CV, save/load, feature importance
engine/race_engine.py	DriverState; pit schedule; single-lap inference; simulation loop
analytics/compare.py	Final classification; position error; lap-time RMSE; gap evolution
analytics/qualifying.py	Best-lap extraction; qualifying prediction; qualifying report
pipeline.py	run_circuit and run_multi_circuit orchestration
run_bahrain / run_spa / run_multi_circuit .py	Thin entry points addressing races by name

Formula Uno — a learned pace oracle wrapped in a deterministic race simulator, validated against a season it was never shown.